

DATA STRUCTURES & ALGORITHMS

COMPREHENSIVE PATTERNS & PROBLEM-SOLVING HANDBOOK

A Unified Reference of Mental Models, Boilerplate Templates, Standard Problems & Code Recipes

Target Domain	Technical Interviews, Product Placements & Coding Screenings
Core Language	Python 3 (Idiomatic, Clean, Production-Grade)
Scope	All 16 DSA Models: Two Pointers, Sliding Window, Backtracking, Trees, Heaps, Graphs, DP & Stack
Source Progress	Compiled directly from repository daily logs, pattern files & core implementations

Ch	Pattern / Algorithmic Model	Key Canonical Questions Covered
1	Python Mechanics & Memory for DSA	Shallow vs Deep Copy, Mutable Recursion Stack, Dict Idioms, Enumerate trap
2	Two Pointers Pattern (4 Archetypes)	LC 125, LC 680, LC 26, LC 283, LC 167, LC 15 (3Sum), LC 345, LC 121, SPLITPAL
3	Fast & Slow Pointers (Floyd's)	LC 141 (Cycle Detect), LC 142 (Cycle Entry), LC 287 (Duplicate Number)
4	Sliding Window (Fixed & Dynamic)	LC 643 (Max Average), LC 3 (Longest Substr), LC 424 (Char Replacement), LC 567
5	Prefix, Suffix & Range Tricks	LC 238 (Product Except Self), LC 303, LC 560 (Subarray Sum K), LC 724, Range Diff
6	Hashing, Signatures & Grouping	LC 387 (First Unique), LC 49 (Group Anagrams), LC 36 (Sudoku), Encode/Decode
7	Stack & Monotonic Stack	Monotonic Decreasing (Next Greater), LC 739 (Daily Temp), LC 155 (MinStack O(1))
8	Queue & Deque Architecture	List O(N) vs Deque O(1), LC 933 (Recent Calls), LC 232 (Queue via Stacks)
9	Binary Search & Search-on-Answer	Standard 1D, LC 74 (2D Matrix), LC 153 (Rotated Min), LC 33, LC 875 (Koko Bananas)
10	Linked Lists Manipulation	LC 143 (Reorder List: Middle -> Reverse -> Alternating Merge)
11	Trees & BST Traversals	BFS Level-Order (LC 102), Balanced Tree (LC 110), Good Nodes, LCA, Validate BST, LC 105
12	Heaps & Priority Queues	Array Indices Math, LC 703 (Kth Largest), LC 295 (Median via 2 Heaps), LC 621
13	Recursion & Backtracking Masterclass	Decision Rules, LC 78, LC 90, LC 39, LC 40, LC 46, LC 22, LC 79, LC 131, LC 51
14	Graphs & Grid Traversals	LC 200 (Islands DFS), LC 994 (Rotting Oranges Multi-BFS), Topological Sorting
15	Dynamic Programming Foundations	Memo vs Tabulation, LC 70 (Climbing Stairs), LC 198 (House Robber), LC 322, LIS vs LCS
16	Sorting Algorithms & Bit Tricks	TimSort Hybrid Intuition, Bucket Sort O(N), LC 136 (Single Number via XOR)

CHAPTER 1 — PYTHON MECHANICS & MEMORY FOUNDATIONS FOR DSA

A solid grasp of Python's execution model and memory semantics is critical to avoiding subtle bugs, Time Limit Exceeded (TLE) errors, and unexpected mutations during coding interviews.

1. Python Reference Model & Copy Semantics

In Python, variables are labels pointing to objects in memory. Simple assignment ($b = a$) never creates a new object; it merely creates an alias pointing to the exact same memory location.

Code Implementation: Shallow vs Deep Copy

```
import copy

# 1. Pointer Assignment (Alias): Mutating 'b' directly alters 'a'
a = [1, 2, 3]
b = a
b.append(4) # a is now [1, 2, 3, 4]

# 2. Shallow Copy: Copies outer container; inner nested objects remain shared!
nested = [[1, 2], [3, 4]]
shallow = copy.copy(nested)
shallow[0].append(99) # nested[0] is now [1, 2, 99]!

# 3. Deep Copy: Recursively duplicates all nested objects independently
deep = copy.deepcopy(nested)
deep[0].append(100) # nested[0] remains untouched!
```

Critical Gotcha: Mutable Objects in Recursive Stack Frames

When passing a mutable list (e.g., `arr`) down recursive calls: each stack frame receives its own local reference to the **same underlying list object** in heap memory. Operations like `arr.append()` alter the shared object across all frames and persist after returning. To isolate state per branch, pass `path.copy()` or explicitly append and pop (Backtracking).

2. Essential Dictionary & Hashing Idioms

Dictionaries are built on hash tables with average $O(1)$ lookup, insertion, and deletion. Avoid calling `.pop()` or `del` inside a loop over list items as it causes $O(N^2)$ degradation.

Code Implementation: Hashing Idioms & Enumerate Trap

```
# Frequency Map Idiom (Clean & Standard)
freq = {}
for num in nums:
    freq[num] = freq.get(num, 0) + 1

# Sorting Dictionary by Value (e.g., Top K Frequent)
# unique.items() yields (key, val) pairs; key=lambda x: x[1] sorts by value
sorted_by_val = sorted(freq.items(), key=lambda x: x[1], reverse=True)

# Finding Key with Max Value
max_key = max(freq, key=freq.get)

# Enumerate Index Trap:
# enumerate(arr[1:]) starts indexing at 0, NOT 1!
# If original indices matter, always use range(1, len(arr)).
```

CHAPTER 2 — TWO POINTERS PATTERN (4 ARCHETYPES)

The Two Pointers pattern utilizes two reference indices traversing an iterable to reduce brute force nested iterations from $O(N^2)$ down to $O(N)$, typically while maintaining $O(1)$ auxiliary space.

When to Use & The 4 Archetypes

- **Converging Pointers:** Pointers start at opposite ends (left = 0, right = n - 1) and move toward each other. Used for sorted pair searches, palindromes, and container capacities.
- **Parallel Pointers:** Pointers move in the same direction at varying speeds or conditions. Left tracks the write/partition boundary; right scans forward.
- **Trigger-based Pointers:** Lead-lag spacing (e.g., finding the k-th node from end of a stream/list).
- **Fast & Slow Pointers:** Pointers advance at different step ratios (1x vs 2x) for cycle detection (Floyd's algorithm).

Standard Problem 1: LC 15 — 3Sum

Find all unique triplets [nums[i], nums[j], nums[k]] such that their sum equals 0. Requires sorting to enable converging two pointers and eliminate duplicate triplets.

Time: $O(N^2)$

Space: $O(1)$ auxiliary

Notes: Sorting takes $O(N \log N)$; two pointers take $O(N)$ for each anchor.

Code Implementation: 3Sum (LC 15)

```
def threeSum(nums: list[int]) -> list[list[int]]:
    nums.sort()
    res = []

    for i in range(len(nums) - 2):
        # Skip duplicate anchors to avoid identical triplets
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        l, r = i + 1, len(nums) - 1
        while l < r:
            total = nums[i] + nums[l] + nums[r]
            if total < 0:
                l += 1
            elif total > 0:
                r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                # Skip duplicate left and right elements
                while l < r and nums[l] == nums[l + 1]:
                    l += 1
                while l < r and nums[r] == nums[r - 1]:
                    r -= 1
                l += 1
                r -= 1

    return res
```

Standard Problem 2: LC 680 — Valid Palindrome II

Determine if a string can be a palindrome after deleting at most one character. Greedy choice: on mismatch at (l, r), check if skipping l OR skipping r creates a palindrome.

Code Implementation: Valid Palindrome II (LC 680)

```
def validPalindrome(s: str) -> bool:
    def is_palindrome_range(i, j):
        while i < j:
            if s[i] != s[j]:
                return False
            i += 1
            j -= 1
        return True

    l, r = 0, len(s) - 1
    while l < r:
        if s[l] == s[r]:
            l += 1
            r -= 1
        else:
            # Skip left char OR skip right char
            return is_palindrome_range(l + 1, r) or is_palindrome_range(l, r - 1)
    return True
```

Standard Problem 3: LC 26 & 283 — In-Place Parallel Pointers**Code Implementation: Parallel Pointers in Action**

```
# LC 26: Remove Duplicates from Sorted Array
def removeDuplicates(nums: list[int]) -> int:
    if not nums: return 0
    write_idx = 1
    for read_idx in range(1, len(nums)):
        if nums[read_idx] != nums[read_idx - 1]:
            nums[write_idx] = nums[read_idx]
            write_idx += 1
    return write_idx

# LC 283: Move Zeroes (In-Place Swap Partition)
def moveZeroes(nums: list[int]) -> None:
    insert_pos = 0
    for i in range(len(nums)):
        if nums[i] != 0:
            nums[insert_pos], nums[i] = nums[i], nums[insert_pos]
            insert_pos += 1
```

CHAPTER 3 — FAST & SLOW POINTERS (FLOYD'S ALGORITHM)

Floyd's Cycle-Finding Algorithm (Tortoise and Hare) uses two pointers moving at different speeds (1 step vs 2 steps). If a cycle exists, the fast pointer will inevitably lap and meet the slow pointer.

Mathematical Intuition: Finding the Cycle Entrance

Let distance from head to cycle entrance = $L1$, entrance to meeting point = $L2$, and cycle length = C .

Slow travelled: $d_slow = L1 + L2$.

Fast travelled: $d_fast = L1 + L2 + k \cdot C = 2 \cdot d_slow$.

Therefore: $L1 + L2 = k \cdot C \Rightarrow L1 = k \cdot C - L2$.

Hence, if we reset slow to head and advance both pointers 1 step at a time, they will meet exactly at the cycle entrance after walking distance $L1$!

Standard Problem 1: LC 142 — Linked List Cycle II

Code Implementation: Linked List Cycle II (LC 142)

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def detectCycle(head: ListNode) -> ListNode:
    slow = fast = head

    # Phase 1: Detect cycle intersection
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            break
    else:
        return None # No cycle exists

    # Phase 2: Find cycle entrance
    slow = head
    while slow != fast:
        slow = slow.next
        fast = fast.next

    return slow
```

Standard Problem 2: LC 287 — Find the Duplicate Number

Given an array of $n + 1$ integers where each integer is between 1 and n . Find the duplicate without modifying the array and using only $O(1)$ extra space. Map array index $i \rightarrow \text{nums}[i]$ as a functional graph!

Time: $O(N)$

Space: $O(1)$

Notes: Guaranteed $O(1)$ space without bitmask or hash table.

Code Implementation: Find Duplicate Number via Cycle Detection (LC 287)

```
def findDuplicate(nums: list[int]) -> int:
    slow = fast = nums[0]

    # Phase 1: Meet inside the cycle
    while True:
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast:
            break

    # Phase 2: Locate entrance (the duplicate value)
    slow = nums[0]
    while slow != fast:
        slow = nums[slow]
        fast = nums[fast]

    return slow
```

CHAPTER 4 — SLIDING WINDOW PATTERN

Sliding Window is a subset of the two-pointer technique applied to contiguous arrays or strings. It maintains a 'window' bounded by $[l, r]$ that dynamically expands and contracts to satisfy problem constraints.

Golden Interview Rule: Why Sliding Window FAILS for Negative Numbers

Sliding window relies strictly on **monotonicity**: expanding the window increases the sum, and shrinking decreases the sum. If the array contains negative numbers, expanding may decrease the sum and shrinking may increase it! For negative numbers with sum targets, **always use Prefix Sum + HashMap ($O(N)$)** instead of sliding window.

1. Fixed Window: LC 643 — Maximum Average Subarray I

Code Implementation: Fixed Window Template (LC 643)

```
def findMaxAverage(nums: list[int], k: int) -> float:
    # Compute initial window sum of size k
    curr_sum = sum(nums[:k])
    max_sum = curr_sum

    # Slide window by 1 element each step
    for i in range(k, len(nums)):
        curr_sum += nums[i] - nums[i - k]
        max_sum = max(max_sum, curr_sum)

    return max_sum / k
```

2. Dynamic Window: LC 3 — Longest Substring Without Repeating Characters

Code Implementation: Dynamic Window with Index Map (LC 3)

```
def lengthOfLongestSubstring(s: str) -> int:
    char_map = {} # stores most recent index of character
    max_len = 0
    l = 0

    for r, char in enumerate(s):
        if char in char_map and char_map[char] >= l:
            # Jump left pointer past previous occurrence
            l = char_map[char] + 1
        char_map[char] = r
        max_len = max(max_len, r - l + 1)

    return max_len
```

3. Advanced Dynamic Window: LC 424 — Character Replacement

Find length of longest substring with same letter after changing at most k characters. Validity condition: $(\text{window_len} - \text{max_freq}) \leq k$.

Code Implementation: Longest Repeating Character Replacement (LC 424)

```
def characterReplacement(s: str, k: int) -> int:
    count = {}
    max_len = 0
    max_freq = 0
    l = 0

    for r in range(len(s)):
        count[s[r]] = count.get(s[r], 0) + 1
        max_freq = max(max_freq, count[s[r]])

        # If invalid, shrink window from left
        while (r - l + 1) - max_freq > k:
            count[s[l]] -= 1
            l += 1

        max_len = max(max_len, r - l + 1)
    return max_len
```

CHAPTER 5 — PREFIX, SUFFIX & RANGE MANIPULATION

Prefix precomputes cumulative information from the start to index i , while Suffix precomputes from the end backwards. Together, they allow queries on ranges or elements without recomputation.

Standard Problem 1: LC 238 — Product of Array Except Self

Return an array where $\text{res}[i]$ equals product of all elements except $\text{nums}[i]$, in $O(N)$ time without division.

Time: $O(N)$

Space: $O(1)$ auxiliary

Notes: Result array excluded from auxiliary space count.

Code Implementation: Product of Array Except Self (LC 238)

```
def productExceptSelf(nums: list[int]) -> list[int]:
    n = len(nums)
    res = [1] * n

    # Pass 1: Prefix products (cumulative product of elements to the left)
    prefix = 1
    for i in range(n):
        res[i] = prefix
        prefix *= nums[i]

    # Pass 2: Suffix products (multiply by cumulative product of elements to right)
    suffix = 1
    for i in range(n - 1, -1, -1):
        res[i] *= suffix
        suffix *= nums[i]

    return res
```

Standard Problem 2: LC 560 — Subarray Sum Equals K (With Negative Numbers)

If $\text{curr_sum} - \text{target} == k$, then the subarray between that prefix and now sums to k ! Store prefix sums in a HashMap with their occurrence frequencies.

Code Implementation: Subarray Sum Equals K (LC 560)

```
def subarraySum(nums: list[int], k: int) -> int:
    prefix_count = {0: 1} # Base case: empty prefix has sum 0
    curr_sum = 0
    count = 0

    for num in nums:
        curr_sum += num
        # If (curr_sum - k) was seen before, add its frequency
        if curr_sum - k in prefix_count:
            count += prefix_count[curr_sum - k]

        prefix_count[curr_sum] = prefix_count.get(curr_sum, 0) + 1

    return count
```

Range Updation Trick (Difference Array)

To execute multiple range update queries $[L, R] += \text{val}$ on an array of size N in $O(1)$ per query instead of $O(N)$:

Code Implementation: Difference Array ($O(1)$ Range Updates)

```
# Difference Array: Add +val at L, subtract -val at R + 1
diff = [0] * (n + 1)
for l, r, val in queries:
    diff[l] += val
    diff[r + 1] -= val

# Reconstruct final array via prefix sum in  $O(N)$  total
for i in range(1, n):
    diff[i] += diff[i - 1]
```

CHAPTER 6 — HASHING, SIGNATURES & CATEGORIZATION

A central problem in algorithmic design is mapping varied inputs to a canonical 'signature' so equivalent items fall into identical buckets in $O(1)$ time.

Standard Problem 1: LC 49 — Group Anagrams

Code Implementation: Group Anagrams via Signature (LC 49)

```
def groupAnagrams(strs: list[str]) -> list[list[str]]:
    groups = {}
    for word in strs:
        # Signature 1: Sorted tuple ( $O(K \log K)$ )
        # Signature 2: 26-element character count tuple ( $O(K)$ )
        count = [0] * 26
        for ch in word:
            count[ord(ch) - ord('a')] += 1
        key = tuple(count)

        if key not in groups:
            groups[key] = []
        groups[key].append(word)

    return list(groups.values())
```

Standard Problem 2: LC 36 — Valid Sudoku

Check validity of a 9x9 Sudoku board in a single pass. Map 3x3 boxes cleanly using coordinate division ($r // 3, c // 3$).

Code Implementation: Valid Sudoku Single-Pass Hash Sets (LC 36)

```
def isValidSudoku(board: list[list[str]]) -> bool:
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]

    for r in range(9):
        for c in range(9):
            val = board[r][c]
            if val == '.':
                continue
            box_idx = (r // 3) * 3 + (c // 3)
            if val in rows[r] or val in cols[c] or val in boxes[box_idx]:
                return False
            rows[r].add(val)
            cols[c].add(val)
            boxes[box_idx].add(val)

    return True
```

Standard Problem 3: String Encode & Decode (NeetCode 150)

Encode a list of strings into a single string without delimiter conflicts by prepending length metadata: $f'\{len(s)}\#\{s\}'$.

Code Implementation: Length-Prefixed String Encoder/Decoder

```
class Codec:
    def encode(self, strs: list[str]) -> str:
        return "".join(f"{len(s)}#{s}" for s in strs)

    def decode(self, s: str) -> list[str]:
        res = []
        i = 0
        while i < len(s):
            j = s.find('#', i)
            length = int(s[i:j])
            res.append(s[j + 1 : j + 1 + length])
            i = j + 1 + length
        return res
```

CHAPTER 7 — STACK & MONOTONIC STACK

A Monotonic Stack maintains its elements in strictly ascending or descending order. As new elements arrive, any elements that violate the ordering are popped and processed.

Golden Rule of Monotonic Stack: Always Store INDICES, Not Values!

Always push **indices** into the monotonic stack rather than raw elements. Storing indices lets you: (1) look up the original values via `nums[stack[-1]]`, (2) compute distances directly via `current_index - stack[-1]`, and (3) update the output array at the exact target positions in $O(1)$.

Monotonic Decreasing Stack: Next Greater Element

Code Implementation: Next Greater Element Template

```
def nextGreaterElements(nums: list[int]) -> list[int]:
    res = [-1] * len(nums)
    stack = [] # stores indices; values are monotonically decreasing

    for i, num in enumerate(nums):
        # Current element is greater than stack top -> resolve top
        while stack and num > nums[stack[-1]]:
            prev_idx = stack.pop()
            res[prev_idx] = num
            stack.append(i)

    return res
```

Standard Problem: LC 739 — Daily Temperatures

Code Implementation: Daily Temperatures (LC 739)

```
def dailyTemperatures(temperatures: list[int]) -> list[int]:
    res = [0] * len(temperatures)
    stack = [] # stores indices

    for i, temp in enumerate(temperatures):
        while stack and temp > temperatures[stack[-1]]:
            prev_idx = stack.pop()
            res[prev_idx] = i - prev_idx # distance in days
            stack.append(i)

    return res
```

Auxiliary Stack Pattern: LC 155 — Min Stack in $O(1)$

Code Implementation: MinStack $O(1)$ All Operations (LC 155)

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val: int) -> None:
        self.stack.append(val)
        current_min = min(val, self.min_stack[-1] if self.min_stack else val)
        self.min_stack.append(current_min)

    def pop(self) -> None:
        self.stack.pop()
        self.min_stack.pop()

    def top(self) -> int:
        return self.stack[-1]

    def getMin(self) -> int:
        return self.min_stack[-1]
```

CHAPTER 8 — QUEUE & DEQUE ARCHITECTURE

Queues operate on First-In-First-Out (FIFO) discipline. In Python, never use standard list as a queue because `list.pop(0)` shifts every subsequent element in memory, causing $O(N)$ degradation.

Why `collections.deque` is $O(1)$ for PopLeft

Python's `collections.deque` is implemented as a doubly-linked list of fixed-size contiguous memory blocks (chunks). Popping from either end simply updates head/tail pointers and deallocates empty blocks in $O(1)$ time without moving elements.

Standard Problem 1: LC 933 — Number of Recent Calls

Track recent pings within a 3000ms window. Evict all expired timestamps from the front of the deque in $O(1)$ amortized time.

Code Implementation: Number of Recent Calls (LC 933)

```
from collections import deque

class RecentCounter:
    def __init__(self):
        self.q = deque()

    def ping(self, t: int) -> int:
        self.q.append(t)
        # Evict timestamps older than t - 3000
        while self.q and self.q[0] < t - 3000:
            self.q.popleft()
        return len(self.q)
```

Standard Problem 2: LC 232 — Implement Queue using Stacks

Code Implementation: Queue Using Two Stacks (Amortized $O(1)$)

```
class MyQueue:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []

    def push(self, x: int) -> None:
        self.in_stack.append(x)

    def pop(self) -> int:
        self.peak()
        return self.out_stack.pop()

    def peek(self) -> int:
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
        return self.out_stack[-1]

    def empty(self) -> bool:
        return not self.in_stack and not self.out_stack
```

CHAPTER 9 — BINARY SEARCH & SEARCH-ON-ANSWER

Binary Search reduces an ordered search space by half at each iteration, achieving logarithmic runtime $O(\log N)$. It applies not just to sorted arrays, but to any monotonic feasibility predicate $P(x)$.

Post-Loop Pointer Semantics: What do 'l' and 'r' mean after the loop?

For standard while $l \leq r$, when the loop terminates with $l > r$:

- l points to the **first element satisfying the condition** (the minimal valid answer / insertion index).
- r points to the **last element violating the condition** (the maximal invalid answer).

Standard Problem 1: LC 153 — Min in Rotated Sorted Array

Code Implementation: Find Minimum in Rotated Sorted Array (LC 153)

```
def findMin(nums: list[int]) -> int:
    l, r = 0, len(nums) - 1
    while l < r:
        mid = (l + r) // 2
        # If mid > right, minimum must lie in the right half
        if nums[mid] > nums[r]:
            l = mid + 1
        else:
            r = mid # mid could be the minimum itself
    return nums[l]
```

Standard Problem 2: LC 33 — Search in Rotated Sorted Array

Code Implementation: Search in Rotated Sorted Array (LC 33)

```
def search(nums: list[int], target: int) -> int:
    l, r = 0, len(nums) - 1
    while l <= r:
        mid = (l + r) // 2
        if nums[mid] == target:
            return mid
        # Left half is sorted
        if nums[l] <= nums[mid]:
            if nums[l] <= target < nums[mid]:
                r = mid - 1
            else:
                l = mid + 1
        # Right half is sorted
        else:
            if nums[mid] < target <= nums[r]:
                l = mid + 1
            else:
                r = mid - 1
    return -1
```

Standard Problem 3: LC 875 — Koko Eating Bananas (Search-on-Answer)

Search space is the speed k from 1 to $\max(\text{piles})$. Predicate: can Koko eat all piles within h hours at speed k ?

Code Implementation: Koko Eating Bananas (LC 875)

```
import math

def minEatingSpeed(piles: list[int], h: int) -> int:
    l, r = 1, max(piles)

    while l <= r:
        k = (l + r) // 2
        total_hours = sum(math.ceil(p / k) for p in piles)

        if total_hours <= h:
            r = k - 1 # Feasible; try smaller speed
        else:
            l = k + 1 # Infeasible; must eat faster

    return l # l holds the minimal feasible speed
```

CHAPTER 10 — LINKED LISTS MANIPULATION

Linked list questions test pointer discipline, sentinel node usage, and cycle-free in-place rearrangement.

Standard Problem: LC 143 — Reorder List

Reorder list from $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$ to $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \dots$ in $O(N)$ time and $O(1)$ space. Combines: (1) Find Middle, (2) Reverse Second Half, (3) Merge Alternately.

Time: $O(N)$

Space: $O(1)$

Notes: In-place pointer rewiring without extra list allocation.

Code Implementation: Reorder List (LC 143)

```
def reorderList(head: ListNode) -> None:
    if not head or not head.next:
        return

    # Step 1: Find middle using slow and fast pointers
    slow, fast = head, head.next
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    # Step 2: Reverse the second half
    second = slow.next
    slow.next = None # Sever connection between halves
    prev = None
    while second:
        nxt = second.next
        second.next = prev
        prev = second
        second = nxt
    second = prev # New head of reversed second half

    # Step 3: Merge two halves alternately
    first = head
    while second:
        tmp1, tmp2 = first.next, second.next
        first.next = second
        second.next = tmp1
        first = tmp1
        second = tmp2
```

CHAPTER 11 — TREES & BINARY SEARCH TREE TRAVERSALS

Tree algorithms decompose into BFS (Level-Order / Queue) and DFS (Call Stack / Recursion: Preorder, Inorder, Postorder). BSTs guarantee that an Inorder Traversal yields strictly sorted values.

1. Level-Order Traversal (BFS): LC 102

Code Implementation: Level-Order BFS Snapshot Pattern

```
def levelOrder(root: TreeNode) -> list[list[int]]:
    if not root: return []
    res = []
    q = deque([root])

    while q:
        level = []
        # Crucial: iterate over fixed snapshot of current level's length
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)

    return res
```

2. DFS Height & Balanced Tree: LC 110

Code Implementation: Balanced Tree Bottom-Up Short Circuit (LC 110)

```
def isBalanced(root: TreeNode) -> bool:
    def check_height(node):
        if not node: return 0
        left = check_height(node.left)
        if left == -1: return -1
        right = check_height(node.right)
        if right == -1: return -1

        if abs(left - right) > 1:
            return -1 # Unbalanced
        return 1 + max(left, right)

    return check_height(root) != -1
```

3. Lowest Common Ancestor in BST: LC 235

Code Implementation: Lowest Common Ancestor in BST (LC 235)

```
def lowestCommonAncestor(root: TreeNode, p: TreeNode, q: TreeNode) -> TreeNode:
    curr = root
    while curr:
        if p.val < curr.val and q.val < curr.val:
            curr = curr.left
        elif p.val > curr.val and q.val > curr.val:
            curr = curr.right
        else:
            return curr # Split point is the LCA!
```

4. Construct Tree from Preorder & Inorder: LC 105

Preorder gives root first. Inorder splits into left and right subtrees. Use a HashMap for O(1) inorder index lookups to avoid O(N²) slicing.

Code Implementation: Construct Binary Tree from Traversals (LC 105)

```
def buildTree(preorder: list[int], inorder: list[int]) -> TreeNode:
    in_map = {val: i for i, val in enumerate(inorder)}
    pre_idx = 0

    def helper(left, right):
        nonlocal pre_idx
        if left > right:
            return None
        root_val = preorder[pre_idx]
        pre_idx += 1
        root = TreeNode(root_val)
        mid = in_map[root_val]

        root.left = helper(left, mid - 1)
        root.right = helper(mid + 1, right)
        return root

    return helper(0, len(inorder) - 1)
```

CHAPTER 12 — HEAPS & PRIORITY QUEUES

A Binary Heap is a complete binary tree backed by an array. Left child is at $2i + 1$, right child at $2i + 2$, and parent at $(i - 1) // 2$.

Heapify $O(N)$ vs Repeated Push $O(N \log N)$

`heapq.heapify(arr)` operates in $O(N)$ time because it sifts down nodes bottom-up: half the nodes are leaves requiring 0 operations, a quarter require 1 swap, and so on. Repeated `heappush` requires $O(N \log N)$.

Standard Problem 1: LC 295 — Find Median from Data Stream

Two-Heaps pattern: max-heap small stores lower half, min-heap large stores upper half. Median is retrieved in $O(1)$.

Code Implementation: Median from Data Stream via Two Heaps (LC 295)

```
import heapq

class MedianFinder:
    def __init__(self):
        self.small = [] # Max-heap (store inverted values)
        self.large = [] # Min-heap

    def addNum(self, num: int) -> None:
        heapq.heappush(self.small, -num)
        # Ensure max element of small <= min element of large
        if self.small and self.large and (-self.small[0] > self.large[0]):
            heapq.heappush(self.large, -heapq.heappop(self.small))

        # Rebalance sizes (small can have at most 1 more element than large)
        if len(self.small) > len(self.large) + 1:
            heapq.heappush(self.large, -heapq.heappop(self.small))
        elif len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def findMedian(self) -> float:
        if len(self.small) > len(self.large):
            return float(-self.small[0])
        return (-self.small[0] + self.large[0]) / 2.0
```

Standard Problem 2: LC 621 — Task Scheduler

Minimizing total time to run tasks with cooldown `n`. Max-heap stores pending task frequencies; queue manages cooldown.

Code Implementation: Task Scheduler (LC 621)

```
from collections import Counter, deque

def leastInterval(tasks: list[str], n: int) -> int:
    counts = Counter(tasks)
    max_heap = [-cnt for cnt in counts.values()]
    heapq.heapify(max_heap)
    q = deque() # stores [remaining_count, available_at_time]
    time = 0

    while max_heap or q:
        time += 1
        if max_heap:
            cnt = 1 + heapq.heappop(max_heap)
            if cnt != 0:
                q.append([cnt, time + n])
        if q and q[0][1] == time:
            heapq.heappush(max_heap, q.popleft()[0])

    return time
```

CHAPTER 13 — RECURSION & BACKTRACKING MASTERCLASS

Backtracking explores state space trees via Depth-First Search. The universal rhythm: **CHOOSE -> EXPLORE (DFS) -> UNDO (BACKTRACK)**.

The 5 Master Decision Rules for Backtracking

1. **i vs i + 1**: Element reusable? Call `dfs(i)` (LC 39). Single use? Call `dfs(i + 1)` (LC 78, 40).
2. **Take/Skip vs For-Loop**: Exactly 2 binary choices per item? Use Take/Don't Take. Multiple choices at position? Use for choice in choices..
3. **Handling Duplicates**: Always sort first. Skip duplicates in skip branch or in loop: if `j > start` and `nums[j] == nums[j-1]`: continue.
4. **When to Save**: Reached end of array (`i == len(nums)`), completed path (`len(path) == len(nums)`), or matched target (`total == target`).
5. **Always copy**: `res.append(path.copy())` because path is modified in-place!

1. Subsets (Take / Don't Take): LC 78 & LC 90

Code Implementation: Subsets I & II Templates

```
# LC 78: Subsets (Take / Skip)
def subsets(nums: list[int]) -> list[list[int]]:
    res, path = [], []
    def dfs(i):
        if i == len(nums):
            res.append(path.copy())
            return
        # TAKE
        path.append(nums[i])
        dfs(i + 1)
        path.pop()
        # DON'T TAKE
        dfs(i + 1)
    dfs(0)
    return res

# LC 90: Subsets II (With Duplicates)
def subsetsWithDup(nums: list[int]) -> list[list[int]]:
    nums.sort()
    res, path = [], []
    def dfs(i):
        if i == len(nums):
            res.append(path.copy())
            return
        path.append(nums[i])
        dfs(i + 1)
        path.pop()
        # Skip all consecutive duplicates in the skip branch
        while i + 1 < len(nums) and nums[i] == nums[i + 1]:
            i += 1
        dfs(i + 1)
    dfs(0)
    return res
```

2. Combination Sum (Reuse vs Single Use): LC 39 & LC 40

Code Implementation: Combination Sum (LC 39)

```
# LC 39: Combination Sum (Reuse allowed: dfs(i, total + nums[i]))
def combinationSum(candidates: list[int], target: int) -> list[list[int]]:
    res, path = [], []
    def dfs(i, total):
        if total == target:
            res.append(path.copy())
            return
        if total > target or i >= len(candidates):
            return
        # TAKE (can reuse current element -> pass i)
        path.append(candidates[i])
        dfs(i, total + candidates[i])
        path.pop()
        # SKIP (move to next -> pass i + 1)
        dfs(i + 1, total)
    dfs(0, 0)
    return res
```

3. Permutations (For-Loop with Used Set): LC 46

Code Implementation: Permutations (LC 46)

```
def permute(nums: list[int]) -> list[list[int]]:
    res, path = [], []
    used = set()
    def dfs():
        if len(path) == len(nums):
            res.append(path.copy())
            return
        for num in nums:
            if num not in used:
                path.append(num)
                used.add(num)
                dfs()
                path.pop()
                used.remove(num)
    dfs()
    return res
```

4. Grid Backtracking: LC 79 — Word Search

Code Implementation: Word Search Grid DFS (LC 79)

```
def exist(board: list[list[str]], word: str) -> bool:
    rows, cols = len(board), len(board[0])
    visited = set()

    def dfs(r, c, k):
        if k == len(word): return True
        if not (0 <= r < rows and 0 <= c < cols) or (r, c) in visited or board[r][c] != word[k]:
            return False
        visited.add((r, c))
        for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
            if dfs(r + dr, c + dc, k + 1):
                return True
        visited.remove((r, c)) # Backtrack
        return False

    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0): return True
    return False
```

5. N-Queens: LC 51

Diagonal invariant: Positive diagonal slope has constant $(r + c)$; Negative diagonal slope has constant $(r - c)$.

Code Implementation: N-Queens Diagonal Mathematical Tracking (LC 51)

```
def solveNQueens(n: int) -> list[list[str]]:
    col_set = set()
    pos_diag = set() # (r + c)
    neg_diag = set() # (r - c)
    res = []
    board = [ "." * n for _ in range(n) ]

    def backtrack(r):
        if r == n:
            res.append("".join(row) for row in board)
            return
        for c in range(n):
            if c in col_set or (r + c) in pos_diag or (r - c) in neg_diag:
                continue
            col_set.add(c); pos_diag.add(r + c); neg_diag.add(r - c)
            board[r][c] = "Q"
            backtrack(r + 1)
            col_set.remove(c); pos_diag.remove(r + c); neg_diag.remove(r - c)
            board[r][c] = "."

    backtrack(0)
    return res
```

CHAPTER 14 — GRAPHS & GRID ALGORITHMS

Graphs represent networks of vertices connected by edges. Matrices can be treated as planar graphs where each cell (r, c) connects to its 4 orthogonal neighbors.

Standard Problem 1: LC 200 — Number of Islands (DFS Sink)

Code Implementation: Number of Islands In-Place Sinking (LC 200)

```
def numIslands(grid: list[list[str]]) -> int:
    if not grid: return 0
    rows, cols = len(grid), len(grid[0])
    islands = 0

    def sink(r, c):
        if not (0 <= r < rows and 0 <= c < cols) or grid[r][c] != '1':
            return
        grid[r][c] = '0' # Sink the land cell
        sink(r + 1, c); sink(r - 1, c); sink(r, c + 1); sink(r, c - 1)

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                islands += 1
                sink(r, c)
    return islands
```

Standard Problem 2: LC 994 — Rotting Oranges (Multi-Source BFS)

Multi-Source BFS starts with all rotten oranges in the queue at time 0 to simulate simultaneous contamination wavefronts.

Code Implementation: Rotting Oranges Multi-Source BFS (LC 994)

```
def orangesRotting(grid: list[list[int]]) -> int:
    rows, cols = len(grid), len(grid[0])
    q = deque()
    fresh = 0

    # Step 1: Enqueue all initial sources and count fresh
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                q.append((r, c))
            elif grid[r][c] == 1:
                fresh += 1

    time = 0
    directions = [(-1,0), (1,0), (0,-1), (0,1)]
    # Step 2: Multi-source BFS propagation
    while q and fresh > 0:
        for _ in range(len(q)):
            r, c = q.popleft()
            for dr, dc in directions:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    q.append((nr, nc))
            time += 1

    return time if fresh == 0 else -1
```

Topological Sorting (DAG Ordering)

Applies only to Directed Acyclic Graphs (DAG). A linear ordering of vertices such that for every directed edge $u \rightarrow v$, u comes before v . Used for prerequisite build orders.

CHAPTER 15 — DYNAMIC PROGRAMMING FOUNDATIONS

Dynamic Programming applies when a problem exhibits **Overlapping Subproblems** and **Optimal Substructure**. It trades memory to eliminate redundant exponential recalculations.

1. State Transition: LC 70 (Climbing Stairs) & LC 198 (House Robber)

Code Implementation: Climbing Stairs & House Robber Tabulation

```
# LC 70: Climbing Stairs (O(1) Rolling Tabulation)
def climbStairs(n: int) -> int:
    one, two = 1, 1
    for _ in range(n - 1):
        one, two = one + two, one
    return one

# LC 198: House Robber
# dp[i] = max(dp[i-1], dp[i-2] + nums[i])
def rob(nums: list[int]) -> int:
    rob1, rob2 = 0, 0
    for num in nums:
        new_rob = max(rob1 + num, rob2)
        rob1 = rob2
        rob2 = new_rob
    return rob2
```

2. Unbounded Knapsack: LC 322 — Coin Change

Code Implementation: Coin Change Bottom-Up DP (LC 322)

```
def coinChange(coins: list[int], amount: int) -> int:
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for a in range(1, amount + 1):
        for c in coins:
            if a - c >= 0:
                dp[a] = min(dp[a], 1 + dp[a - c])

    return dp[amount] if dp[amount] != float('inf') else -1
```

Concept Distinction: LIS vs LCS

- **LIS (Longest Increasing Subsequence):** Finds length of strictly ascending subsequence in a single array (e.g., [10, 22, 9, 33] -> [10, 22, 33], len 3).
- **LCS (Longest Common Subsequence):** Finds longest sequence of elements shared between two separate arrays in the same relative order.

CHAPTER 16 — SORTING ALGORITHMS & BIT MANIPULATION

Sorting Mechanics: TimSort & Bucket Sort

- **TimSort:** Python's built-in `.sort()` and `sorted()`. It divides the array into sorted runs, uses Insertion Sort for small blocks (< 32 elements), and combines them with Merge Sort. Guaranteed $O(N \log N)$ worst-case runtime and $O(N)$ best-case on nearly sorted data.
- **Bucket Sort:** Distributes elements across discrete buckets based on value range or frequency. When the range is small and uniform, sorting finishes in linear $O(N)$ time.

Bit Manipulation: LC 136 — Single Number

Given an array where every element appears twice except for one unique element. Find that element in $O(N)$ time and $O(1)$ space using the XOR cancellation property: $x \oplus x = 0$ and $x \oplus 0 = x$.

Code Implementation: Single Number via XOR Cancellation (LC 136)

```
def singleNumber(nums: list[int]) -> int:
    res = 0
    for num in nums:
        res ^= num # Identical duplicate pairs cancel out to 0
    return res
```

Placement Revision Checklist & Fast Decision Matrix

1. Sorted Array? -> Two Pointers or Binary Search.
2. Substring / Subarray with Min/Max/At-Most-K? -> Sliding Window (Positives only) or Prefix Sum + Map.
3. Next Greater / Next Smaller Element? -> Monotonic Stack (Store Indices!).
4. Kth Largest / Stream Median / Priority Scheduling? -> Heap / Priority Queue.
5. Tree Level-by-Level? -> BFS with Queue + `len(q)` inner snapshot loop.
6. Tree Depth / Ancestors / Paths? -> DFS with recursion.
7. All Combinations / Permutations / Subsets? -> Backtracking (Choose -> Explore -> Undo).
8. Overlapping Calculations with Subproblems? -> Dynamic Programming (Tabulation/Memoization).